

3.5 — Apply Iterative Refinement Techniques for Progressive Improvement

Concept

One of the most reliable ways to get high-quality output from Claude is to stop expecting perfection on the first attempt. Iterative refinement treats Claude as a collaborator in a feedback loop — each exchange narrows the gap between what you received and what you need.

Domain mental model: Think of refinement as gradient descent. You start at a rough approximation, measure the distance from your target, and adjust. The key insight is that the measurement — the feedback — is more valuable than the initial output.

Core Techniques

1. Progressive improvement — start coarse, refine toward precision

Request a first draft explicitly framed as a starting point, not a final answer. This shifts the interaction from "get it right once" to "converge together." A simple framing:

"Give me a rough first pass at this function. We'll refine it."

This lowers the model's tendency to over-hedge or over-explain, and gives you a concrete artifact to react to rather than a blank page.

2. Input/output examples as a refinement anchor

Abstract descriptions of desired behavior are ambiguous. Concrete input/output pairs are not. When you provide expected inputs alongside their expected outputs, you give Claude an objective test it can reason against during generation.

"When the input is `{status: 'active', role: 'viewer'}`, the output should be `{canEdit: false, canView: true}`."

The more examples you provide — especially edge cases — the more precisely Claude can calibrate. This is the prompting equivalent of a unit test suite: each example rules out a class of incorrect implementations.

3. Test-driven iteration — write the tests first, then implement

Rather than asking for an implementation and then checking whether it passes your tests, give Claude the tests as part of the prompt. Instruct it to write an implementation that satisfies them. This mirrors TDD:

1. Provide failing test cases (RED)
2. Ask for the implementation (GREEN)
3. Review and ask for refactoring if needed (IMPROVE)

If the implementation fails a test, feed the failure back to Claude verbatim. It can reason about the exact mismatch and correct it without requiring you to restate the full context.

4. The interview pattern — clarify before implementing

For ambiguous tasks, instruct Claude to ask clarifying questions before writing any code. This prevents wasted effort on implementations that miss unstated requirements.

"Before implementing, identify any ambiguities and ask me to resolve them. Only proceed once you have enough information."

This is especially valuable when the task involves non-obvious constraints (performance budgets, data shape assumptions, side-effect boundaries). A well-structured clarification round often halves the number of revision cycles needed.

5. Structured feedback loops — name what changed

When iterating, be explicit about what was wrong in the previous output and what the desired change is. Vague feedback ("make it better") produces vague improvements. Precise feedback produces targeted corrections:

"The previous version creates a new array on every call. Refactor to reuse the existing buffer."

Code Example

Scenario: You need a function that normalizes a user object before writing to the database. Requirements are partially unclear.

Round 1 — Interview pattern

Prompt:

```
"I need a function that normalizes a user object before a database write.  
Before implementing, list any ambiguities you'd want resolved."
```

Claude asks:

1. Should fields be trimmed of whitespace?
2. Should missing optional fields be set to null or omitted?
3. Should email be lowercased?
4. How should unknown fields be handled — stripped or passed through?

You answer each question. Now Claude implements with full context.

Round 2 — Input/output examples as specification

Prompt:

"Implement `normalizeUser`. Use these examples as your spec:

Input: { name: " Alice ", email: "Alice@Example.COM", age: 30, unknown: "x" }

Output: { name: "Alice", email: "alice@example.com", age: 30 }

Input: { name: "Bob", email: "bob@test.com" }

Output: { name: "Bob", email: "bob@test.com" }

"

Round 3 — Test-driven correction

Prompt:

"The following test is failing:

```
expect(normalizeUser({ name: " " })).toEqual({ name: "" })
```

```
// Received: { name: " " }
```

Fix the implementation so trimming applies even when the result is an empty string."

Three focused rounds. Each narrows the implementation toward the exact specification without restarting from scratch.

Common Mistakes

- 1. Demanding perfection in the first prompt** Overloading a prompt with every edge case and constraint at once produces bloated, defensively-written output. Lead with the core case, then layer in edge cases during refinement.
- 2. Providing vague feedback** "This isn't quite right" gives Claude no signal about which part failed or why. Always specify: what the output did, what you expected, and what the difference is.
- 3. Restating the full context on every iteration** Claude retains the conversation. When refining, refer to the previous output by reference ("in the previous version") rather than repeating the full specification. This saves context and avoids introducing inconsistencies.
- 4. Skipping the interview pattern for ambiguous tasks** Jumping straight to implementation when requirements are underspecified guarantees rework. Spending one exchange on clarification is almost always faster than two or three revision cycles.
- 5. Treating the first test failure as a reason to rewrite from scratch** A single failing test usually indicates a localized defect. Feed the exact failure back rather than discarding the implementation. Targeted correction preserves the parts that already work.
- 6. Forgetting to close the loop** After a correction, verify the fix explicitly. Ask Claude to re-evaluate the corrected implementation against all prior examples or tests, not just the one that failed.

References

- Anthropic prompt engineering guide: Iterative refinement and few-shot prompting
- Exam domain 3: Prompt Engineering and Interaction Design
- Related handouts: 3.1 (Prompt Structure), 3.3 (Few-Shot Examples), 3.4 (Chain-of-Thought)
- See also: Test-driven development principles — the RED/GREEN/REFACTOR cycle maps directly onto multi-turn prompt refinement